

Programowanie Aplikacji w Chmurze Obliczeniowej

Laboratorium nr 13D+14

Konfiguracja danych wrażliwych w Docker Compose. Wykorzystanie Secrets.
Docker Compose w połączenie z łańcuchami CI/CD. Mechanizm merge.

Dr inż. Sławomir Przyłucki
s.przylucki@pollub.pl





Docker Compose - dane wrażliwe - cz. I

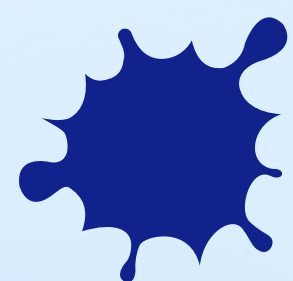


Środowisko Docker zawiera rozwiązanie do zarządzania danymi wrażliwymi (sekretami), ale nie działają one z samodzielnymi kontenerami.

Ich wykorzystanie ograniczone jest do Docker Compose (ograniczona funkcjonalność) oraz do środowiska klastrowego Docker Swarm.

Dostanie sekretu do kontenera w ramach pliku docker-compose.yaml jest procesem dwuetapowym.

1. Należy zdefiniować tzw. *top-level secrets element* opisujący powiązanie danego secret ze źródłem danych wrażliwych
2. Za pomocą atrybutu `secrets` należy zaktualizować definicję tych serwisów wchodzących w skład aplikacji, które wykorzystują dane wrażliwe. Aktualizacja polega typowo na powiązaniu zmiennej reprezentującą określone dane wrażliwe z sekretem opisanym wg. punktu 1.



Docker Compose zapewnia najprostszy sposób na korzystanie z sekretów **bez konieczności używania zmiennych środowiskowych** do przechowywania wrażliwych informacji.

<https://docs.docker.com/compose/use-secrets/>



Docker Compose - dane wrażliwe - cz. II

PRZYKŁAD pochodzi z:

<https://docs.docker.com/compose/how-tos/use-secrets/#examples>

```
1  services:
   |   > Run Service
2  |   db:
3  |     image: mysql:latest
4  |     volumes:
5  |       - db_data:/var/lib/mysql
6  |     environment:
7  |       MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_root_password
8  |       MYSQL_DATABASE: wordpress
9  |       MYSQL_USER: wordpress
10 |       MYSQL_PASSWORD_FILE: /run/secrets/db_password
11 |     secrets:
12 |       - db_root_password
13 |       - db_password
```

Sekrety są montowane w predefiniowanej ścieżce kontenera: `/run/secrets/<secret_name>`.

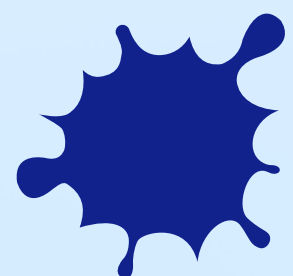
top-level secrets element

Sekcja `secrets` definiuje dwa secret-y o nazwach, odpowiednio: `db_root_password` i `db_password`

```
29  secrets:
30  |   db_password:
31  |     file: db_password.txt
32  |   db_root_password:
33  |     file: db_root_password.txt
```

Wartość secret-ów jest odczytywana z plików dostępnych w katalogu roboczym (w katalogu, w którym jest plik dla docker-compose)

<https://docs.docker.com/compose/compose-file/09-secrets/>

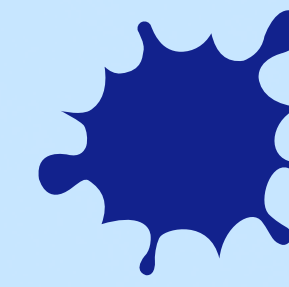


Po uruchomieniu kontenera zawartość plików zawierających dane wrażliwe jest zostanie odczytywana i montowana (bind mount) w systemie plików kontenera, w miejscu wskazanym w pliku `docker-compose`



PAwChO — Laboratorium 13D

Przykład wykorzystania secrets



Przykładowe pliki dostępne na moode: **lab13D14_examples.zip**

```
~/examples/compose2
> docker compose -f docker-compose.secrets.yml up -d
[+] up 43/43
✓ Image wordpress:latest Pulled
✓ Image mysql:latest Pulled
✓ Network compose2_default Created
✓ Volume compose2_db_data Created
✓ Container compose2-db-1 Started
✓ Container compose2-wordpress-1 Started
```

Uruchomienie na bazie pliku `docker-compose.secrets.yml`

Potwierdzenie poprawnego uruchomienia aplikacji

```
~/examples/compose2
> docker compose -f docker-compose.secrets.yml ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
compose2-db-1	mysql:latest	"docker-entrypoint.s..."	db	About a minute ago	Up About a minute	3306/tcp, 33060/tcp
compose2-wordpress-1	wordpress:latest	"docker-entrypoint.s..."	wordpress	About a minute ago	Up About a minute	0.0.0.0:8000->80/tcp, [::]:8000->80/tcp

argocd on argocd (default)

Potwierdzenie, że sekret został powiązany z serwisem za pomocą mechanizmu *bind mounts*

```
~/examples/compose2
> docker container inspect compose2-db-1 | jq '.[].Mounts'
[
  {
    "Type": "volume",
    "Name": "compose2_db_data",
    "Source": "/var/lib/docker/volumes/compose2_db_data/_data",
    "Destination": "/var/lib/mysql",
    "Driver": "local",
    "Mode": "rw",
    "RW": true,
    "Propagation": ""
  },
  {
    "Type": "bind",
    "Source": "/Users/slawek/examples/compose2/db_root_password.txt",
    "Destination": "/run/secrets/db_root_password",
    "Mode": "",
    "RW": false,
    "Propagation": "rprivate"
  },
  {
    "Type": "bind",
    "Source": "/Users/slawek/examples/compose2/db_password.txt",
    "Destination": "/run/secrets/db_password",
    "Mode": "",
    "RW": false,
    "Propagation": "rprivate"
  }
]
```



Zdecydowanie bardziej zaawansowane rozwiązania (zgodne z zaleceniami 12 Factors) są dostępne w klastrze dla środowiska Docker - Docker Swarm oraz ... oczywiście w klastrze Kubernetes



PAwChO — Laboratorium 13D

Zadanie - część NIEOBOWIĄZKOWA

Należy zmodyfikować rozwiązanie zadania obowiązkowego z lab. 12 w następujący sposób:

- Wszystkie dane wykorzystywane przez aplikację LEMP, które mogą być uznane za wrażliwe, należy skonfigurować jako secrets (analogicznie jak przykłady z pierwszej części tej instrukcji)



W sprawozdaniu proszę umieścić zmodyfikowane pliki konfiguracyjne Compose. W piku README.md należy podać wszystkie użyte polecenia wraz z wynikiem ich działania. Dodatkowo, należy dowieść, że po wprowadzonych modyfikacjach :

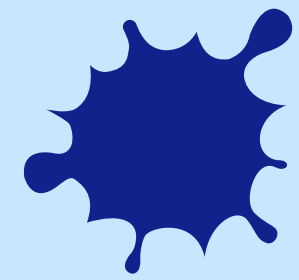
- stack LEMP działa poprawnie i wyświetla domyślnie wymaganą stronę,
- można zainicjować testową bazę danych

Preferowana forma sprawozdania - link do repo na GitHub.



Docker Compose - wykorzystanie w łańcuchach CI/CD

W przypadku wykorzystania Docker Compose w łańcuchach CI/CD podstawowym zagadnieniem jest możliwość sprawnej i zautomatyzowanej zmiany parametrów aplikacji wielokontenerowej tak by dopasować je do określonego środowiska (development, testing, staging itd.)



Omówienie wykorzystania Docker Compose w środowiskach produkcyjnych:

<https://docs.docker.com/compose/production/>

W kontekście wykorzystania w łańcuchach CI/CD dostępne są trzy podstawowe mechanizmy:

Merge - Docker Compose umożliwia scalanie i nadpisanie w ramach zestawu plików w celu utworzenia jednego, złożonego pliku Compose.

Extend - mechanizm ten pozwala na współdzielenie konfiguracji między różnymi plikami Compose, a nawet całkowicie różnymi projektami. Pozwala na wskazania, które zapisy z innych plików Compose mają być użyte we własnej aplikacji oraz daje możliwość nadpisania niektórych atrybutów.

Include - umożliwia modularyzację projektów poprzez możliwość włączania oddzielnych plików Compose bezpośrednio do bieżącego pliku *compose.yaml*.

<https://docs.docker.com/compose/multiple-compose-files/>



Docker Compose - mechanizm merge

Idea mechanizmu merge polega na umożliwieniu scalania i nadpisanie zestawu plików Compose w celu utworzenia złożonego, wynikowego pliku Compose.

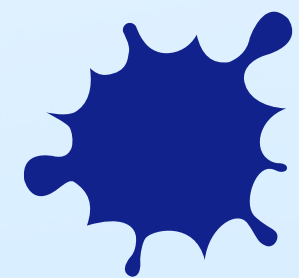


⚠ Important

When you use multiple Compose files, you must make sure all paths in the files are relative to the base Compose file (the first Compose file specified with `-f`). This is required because override files need not be valid Compose files. Override files can contain small fragments of configuration.

Tracking which fragment of a service is relative to which path is difficult and confusing, so to keep paths easier to understand, all paths must be defined relative to the base file.

<https://docs.docker.com/compose/multiple-compose-files/merge/#how-to-merge-multiple-compose-files>



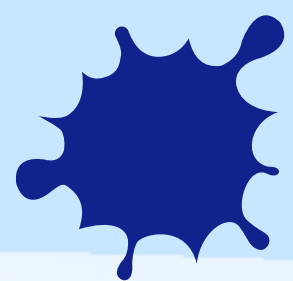
Jeśli usługa jest zdefiniowana w więcej niż jednym pliku, Compose łączy konfiguracje przy użyciu reguł opisanych w specyfikacji Compose.

<https://docs.docker.com/compose/compose-file/13-merge/>



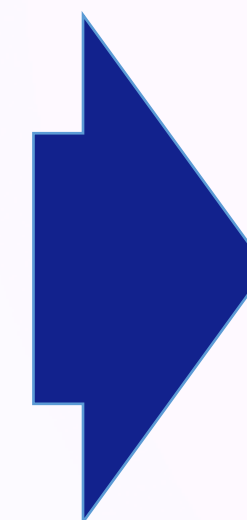
Mechanizm merge - przykład - cz. I

Przykładowo, można zdefiniować podstawowy plik Docker Compose, a następnie zdefiniować tzw. plik override zawierający definicje atrybutów aplikacji specyficznych dla danego środowiska docelowego.



Plik bazowy powinien definiować tylko te składniki aplikacji, które powinny być takie same we wszystkich środowiskach. Wszystkie konkretne (specyficzne) ustawienia powinny zostać usunięte.

Przykładowy plik bazowy:
`dockercompose.base.yml`



```
1  services:
   ▸ Run Service
2  db:
3    image: postgres:alpine
4    volumes:
5      - pg-data:/var/lib/postgresql/data
6      - ./db:/docker-entrypoint-initdb.d
7
8  ▸ Run Service
9  pgadmin:
10    image: dpage/pgadmin4
11    volumes:
12      - pgadmin-data:/var/lib/pgadmin
13
14  ▸ Run Service
15  web:
16    image: spg51/labx:web
17    depends_on:
18      - db
19
20  volumes:
21    pg-data:
22    pgadmin-data:
```



Mechanizm merge - przykład - cz. II

```
1  services:
    ▷ Run Service
2  db:
3    environment:
4      - POSTGRES_USER=dockeruser
5      - POSTGRES_PASSWORD=dockerpass
6      - POSTGRES_DB=pets
7
8    ▷ Run Service
9  pgadmin:
10   ports:
11     - 8080:80
12   environment:
13     PGADMIN_DEFAULT_EMAIL: ci-admin@dcompose.lab
14     PGADMIN_DEFAULT_PASSWORD: ci-admin
15
16   ▷ Run Service
17 web:
18   ports:
19     - 80:3000
20   environment:
21     - DB_HOST=db
```

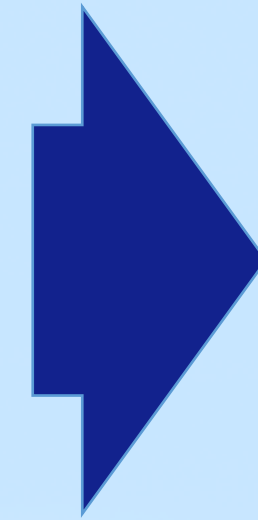
Założmy, że należy uruchomić aplikację w ramach łańcucha CI. W takim przypadku należy utworzyć odpowiedni plik override, np. `docker-compose.ci.yml` i dodać do niego opisy ustawień aplikacji dla danego środowiska docelowego.

Przykładowy plik override:
`dockercompose.ci.yml`



Mechanizm merge - przykład - cz. III

Uruchomienie
przykładowej aplikacji:



```
~/examples/compose2
> docker compose -f docker-compose.base.yml -f docker-compose.ci.yml up -d
[+] up 41/41
  ✓ Image dpage/pgadmin4 Pulled
[+] up 46/4651/labx:web Pulled
  ✓ Image dpage/pgadmin4 Pulled
  ✓ Image spg51/labx:web Pulled
  ✓ Image postgres:alpine Pulled
  ✓ Volume compose2_pg-data Created
  ✓ Volume compose2_pgadmin-data Created
  ✓ Container compose2-pgadmin-1 Started
  ✓ Container compose2-db-1 Started
  ✓ Container compose2-web-1 Started
```



Standardowa konwencją nazewnictwa dla mechanizmu merge:

- plik bazowy: `docker-compose.yml`
- plik nadpisywania: `docker-compose.override.yml`,

Stosując to nazewnictwo można pomijać nazwy plików w poleceniu uruchamiającym aplikację (brak opcji `-f`)

Potwierdzenie działania wdrożonej aplikacji:



```
~/examples/compose2
> docker compose ls
NAME          STATUS      CONFIG FILES
compose2      running(3)  /Users/slawek/examples/compose2/docker-compose.base.yml,/Users/slawek/examples/compose2/docker-compose.ci.yml,/Users/slawek/examples/compose2/docker-compose.secrets.yml
~/examples/compose2
> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
000b6943385e   spg51/labx:web "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  0.0.0.0:80->3000/tcp, [::]:80->3000/tcp  compose2-web-1
e4271055982f   dpage/pgadmin4 "/entrypoint.sh"         2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  compose2-pgadmin-1
29db5899d3af   wordpress:latest "docker-entrypoint.s..." 11 minutes ago Up 11 minutes  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp  compose2-wordpress-1
```



Zadanie DODATKOWE - wyłącznie w celu poprawy oceny

Należy zmodyfikować rozwiązanie zadania NIEOBOWIĄZKOWEGO z lab. 13D w następujący sposób:

- Należy zaproponować podział pliku docker-compose.yaml na plik bazowy i plik override (analogicznie jak w drugiej części laboratorium)



W sprawozdaniu proszę umieścić zmodyfikowane pliki konfiguracyjne Compose. W pliku README.md należy podać wszystkie użyte polecenia wraz z wynikiem ich działania. Dodatkowo, należy dowieść, że po wprowadzonych modyfikacjach :

- stack LEMP działa poprawnie i wyświetla domyślnie wymaganą stronę,
- można zainicjować testową bazę danych

Preferowana forma sprawozdania - prezentacja rozwiązania na ostatnim laboratorium